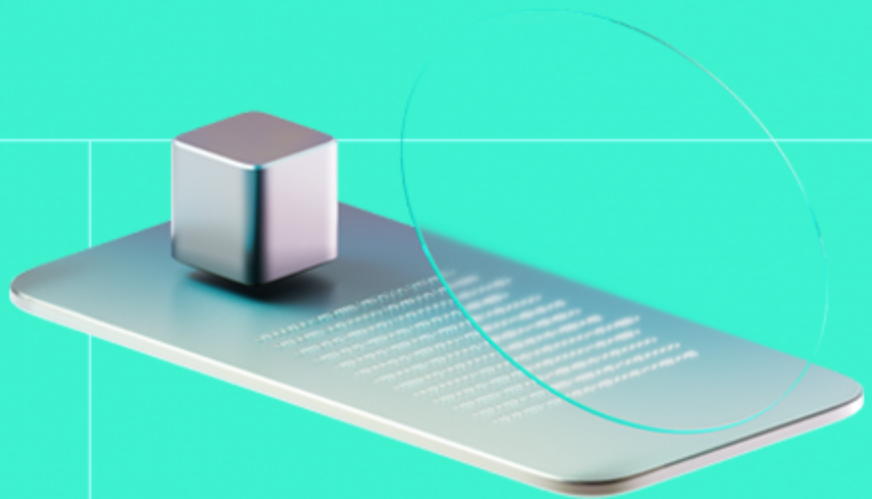




Smart Contract Code Review And Security Analysis Report

Customer: Node Meta

Date: 07/03/2026



We express our gratitude to the Node Meta team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

Enterprise-grade BEP20 token with advanced DeFi features, MEV protection, and emergency controls.

Document

Name	Smart Contract Code Review and Security Analysis Report for Node Meta
Audited By	Khrystyna Tkachuk, Kerem Solmaz
Approved By	Ivan Bondar, Turgay Arda Usman
Website	https://node-meta.com/
Changelog	11/02/2026 - Preliminary Report
	24/02/2026 - Interim Report
	07/03/2026 - Final Report
Platform	BSC
Language	Solidity
Tags	ERC20, Fee-On-Transfer, Signatures
Methodology	https://docs.hacken.io/methodologies/smart-contracts

Review Scope

Repository	https://github.com/nodemeta/nte
Initial Commit	04d20e37cacc93e42945ac691a297688545e9b3a
Final Commit	58ae668b2b8075c0d5f6a54f87036a464e305068

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

20	18	1	1
Total Findings	Resolved	Accepted	Mitigated

Findings by Severity

Severity	Count
Critical	0
High	1
Medium	5
Low	1

Vulnerability	Severity	Status
F-2026-15019 - Double Taxation on Liquidity ETH Removal via Router	High	Fixed
F-2026-14942 - Incorrect maxSellAmount Calculation Allows Selling Up to Total Supply Amount	Medium	Fixed
F-2026-14988 - pancakeRouter and pancakePair Initialization Can Be Skipped in Constructor With No Recovery Mechanism	Medium	Fixed
F-2026-15004 - Hardcoded Primary Pair In _calculatePriceImpact() Leads To Price Impact Limit Bypass On Secondary DEX Pairs	Medium	Fixed
F-2026-15018 - Buy and Sell Fees Unexpectedly Applied During Liquidity Provision Operations	Medium	Mitigated
F-2026-15022 - Owner Rights Can Be Renounced While Contract Is Paused Permanently Locking Transfer	Medium	Fixed
F-2026-14955 - Missing Expiration Deadline In TransactionFrom Signature Leads To Indefinitely Valid Signed Authorizations	Low	Fixed
F-2026-14943 - Floating Pragma	Info	Fixed
F-2026-14953 - Semantically Incorrect Error Codes Across Multiple Functions Lead To Misleading Revert	Info	Fixed
F-2026-14956 - Missing Early Input Validation In TransactionFrom Leads To Unnecessary Gas Consumption On Invalid Calls	Info	Fixed
F-2026-14989 - Unused Event and Custom Error Declaration	Info	Fixed
F-2026-14990 - Lack of Event Emitting for Key State Changes	Info	Fixed
F-2026-14992 - Redundant Assignment of Default Values	Info	Fixed
F-2026-14993 - Inconsistent Usage of Naming Conventions	Info	Fixed

Vulnerability	Severity	Status
F-2026-14994 - State Variables Can Be Marked as Immutable	Info	Fixed
F-2026-15007 - Inconsistent Use Of Magic Numbers Across Tax And Protection Configuration Reduces Maintainability And Increases Risk Of Divergence	Info	Fixed
F-2026-15020 - Public Function not Used Internally can be Marked External	Info	Fixed
F-2026-15021 - Lack of Two Step Ownership Transfer Mechanism	Info	Fixed
F-2026-15023 - Redundant receive Function	Info	Accepted
F-2026-15041 - Missing msg.sender Validation in _transferWithTax	Info	Fixed

Documentation quality

- Functional requirements are partially missed.
- Technical description is not provided.

Code quality

- The code duplicates commonly known contracts instead of reusing them.
- NatSpec is included across the contracts.
- The development environment is configured.

Test coverage

Code coverage of the project is **0 %**.

- Tests are not provided.

Table of Contents

System Overview	7
Privileged Roles	7
Potential Risks	
Findings	9
Vulnerability Details	9
Disclaimers	56
Appendix 1. Definitions	57
Severities	57
Potential Risks	57
Appendix 2. Scope	58

System Overview

NTE is an advanced **BEP20** token (BNB Chain) that extends standard ERC20 with:

- **Configurable taxes** on buy, sell, and P2P transfers, with a dedicated treasury and optional auto-liquidity routing.
- **PancakeSwap V2 integration** for a canonical NTE/WETH pair and DEX-aware tax (buy vs sell vs transfer).
- **Security and anti-abuse** layers: pause, anti-bot window, blacklist/whitelist, MEV protection, velocity limits, anti-dump, price-impact cap, and wallet cooldown.
- **Categorized transfers** with off-chain signatures (auth signers) for business/reward flows.
- **Staking integration** via a designated staking contract that can lock/unlock user balances.

No minting after deployment; only the initial supply is minted in the constructor. Any address can burn its own tokens.

It has the following attributes:

- Name: Node Meta Energy
- Symbol: NTE
- Decimals: 18
- Total supply: defined on the contract deployment.

Privileged roles

The owner of the NTE contract has administrative control over the contract configuration, including pausing functionality, ownership management, tax updates, blacklist and whitelist management, and DeFi protection mechanisms (`setAntiDumpConfig` , `setPriceImpactLimitConfig` , `setPriceImpactExempt` , `setWalletCooldownConfig` , `setMevProtectionConfig` , `setMevProtectionExempt` , `setVelocityLimitConfig` , `setVelocityLimitExempt`). The owner can also enable or disable the DEX pair address, set the `stakingContract` address, manage `authSigner` roles, manage payment categories (`addCategory` , `setCategoryEnabled` , `updateCategoryName`), and perform emergency withdrawals of ERC20 tokens and BNB. Emergency ERC20 token withdrawal has no time restriction, while BNB withdrawal requires 30 days post-launch. Ownership can be transferred instantly, but renouncing ownership is locked for 30 days after deployment. The owner is exempt from taxes and is not subject to anti-bot restrictions. If `pauseIncludesOwner` is set to `true` , even the owner's transfers can be blocked when the contract is paused.

The authorized signers stored in the `isAuthSigner` mapping are off-chain signers whose signatures are required to authorize categorized transfers. Any caller with sufficient allowance from the `from` address can execute `TransactionFrom()` , provided they supply a valid signature from an authorized signer.

The `stakingContract` can lock (`lockFromStaking()`) or unlock (`unlockFromStaking()`) tokens on behalf of users. The locked balance is enforced within `_transfer()` : a user cannot transfer tokens if doing so would reduce their balance below `LockedForStaking[user]`

Potential Risks

- **Fixed Total Supply Post-Deployment:** The token's total supply is determined at deployment and cannot be verified beforehand, potentially limiting the project's adaptability and economic model flexibility.
- **Centralized Minting to a Single Address:** The project concentrates minting tokens in a single address, raising the risk of fund mismanagement or theft, especially if key storage security is compromised.
- **Owner's Unrestricted State Modification:** The absence of restrictions on state variable modifications by the owner leads to arbitrary changes, affecting contract integrity and user trust, especially during critical operations like minting phases.
- **Single Points of Failure and Control:** The project is fully or partially centralized, introducing single points of failure and control. This centralization can lead to vulnerabilities in decision-making and operational processes, making the system more susceptible to targeted attacks or manipulation.
- **Dependency on External Logic for Implemented Logic:** The implemented NTE logic depends on `stakingContract` implementation not covered by the audit that calls `lockFromStaking` and `unlockFromStaking` functions. This reliance introduces risks if these external contracts are compromised or contain vulnerabilities, affecting the audited project's integrity.
- **Interactions with External DeFi Protocols:** The NTE contract depends on PancakeSwap (Uniswap V2) for price impact calculations, pair detection, and tax routing. If the PancakeSwap router or pair contracts are compromised or behave unexpectedly, it could affect tax calculations, MEV protection logic, and anti-dump enforcement. This is distinct from your existing staking dependency risk.
- **Absence of Time-lock Mechanisms for Critical Operations:** The owner can instantly change critical parameters like blacklisting/unblacklisting addresses, toggling pause, changing treasury address, modifying MEV/anti-dump/cooldown settings, and setting DEX pair status - all without any timelock or delay. The only rate-limited operation is tax rate changes (24h cooldown, max 2.5% delta). Users have no advance notice of other impactful changes.
- **Blacklist Capability as Address Freezing Risk:** The owner can permanently blacklist any address (except themselves and the contract), effectively freezing user funds with no recourse. While this can serve compliance purposes, it introduces censorship risk and potential for fund lockup if the owner key is compromised or misused.
- **Owner Tax Exemption and Transfer Privilege:** The owner address is hardcoded as exempt from all taxes, pause restrictions (when `pauseIncludesOwner` is false), anti-bot checks, whitelist requirements, and anti-dump enforcement. This creates a trust requirement where the owner can trade freely under conditions where all other users are restricted.

Findings

Vulnerability Details

[F-2026-15019](#) - Double Taxation on Liquidity ETH Removal via Router - High

Description:

The **NTE** token implements two distinct taxation mechanisms:

- **AMM interaction tax:** applied to buy/sell operations involving the liquidity pair.
- **Transfer tax:** applied to direct token transfers between externally owned accounts (EOAs).

```
function _calculateTax(address from, address to, uint256 amount) private view
returns (uint256) {
    uint256 taxBps = 0;

    if (isPancakePair[from] && !isPancakePair[to]) {
        // Buy: from DEX to user
        taxBps = buyTaxBps;
    } else if (!isPancakePair[from] && isPancakePair[to]) {
        // Sell: from user to DEX
        taxBps = sellTaxBps;
    } else if (isPancakePair[from] && isPancakePair[to]) {
        // Pool-to-pool move (usually arbitrage)
        taxBps = sellTaxBps;
    } else {
        // Just a regular P2P transfer
        taxBps = transferTaxBps;
    }

    if (taxBps == 0) return 0;

    // Check for math overflow before we calculate
    if (amount > type(uint256).max / taxBps) revert TXN_OVERFLOW();

    return (amount * taxBps) / BASIS_POINTS;
}
```

During a **liquidity ETH removal** operation executed via the DEX router, the current tax logic unintentionally applies both taxes, resulting in double taxation for a single user action.

Specifically, when liquidity is removed:

- Tokens are transferred from the **liquidity pair to the router**, where the transaction is interpreted as an AMM buy and the **buy tax** is applied.

```
function removeLiquidityETHSupportingFeeOnTransferTokens(
    ...
) public virtual override ensure(deadline) returns (uint amountToken, uint amountETH) {
    // The first transfer triggered from pair to router
    (, amountETH) = removeLiquidity(
        token,
        WETH,
        liquidity,
        amountTokenMin,
        amountETHMin,
        address(this), // <-- router address passed as recipient of the tokens from pair
        deadline
    );
    ...
}

function removeLiquidity(
    ...
) public virtual override ensure(deadline) returns (uint amountA, uint amountB) {
    address pair = PancakeLibrary.pairFor(factory, tokenA, tokenB);
    IPancakePair(pair).transferFrom(msg.sender, pair, liquidity); // send liquidity to pair
    (uint amount0, uint amount1) = IPancakePair(pair).burn(to); // transfer() from pair to router occurs
    ...
}
```

- Tokens are subsequently transferred from the **router to the end user**, where the transfer is treated as a regular token transfer and the **transfer tax** is applied again.

```
function removeLiquidityETHSupportingFeeOnTransferTokens(
    ...
) public virtual override ensure(deadline) returns (uint amountToken, uint amountETH) {
    ...
    // the second transfer from router to user <to>
    TransferHelper.safeTransfer(token, to, IERC20(token).balanceOf(address(this)));
}
```

```
...  
}
```

As a result, liquidity removal transactions are taxed twice, despite representing a single economic operation initiated by the user.

It may lead to:

- Users removing liquidity receive significantly fewer tokens than expected due to compounded taxation.
- Liquidity providers are economically penalized, discouraging participation and reducing protocol liquidity.
- The effective tax rate during liquidity removal may exceed intended limits, potentially breaking economic assumptions of the tokenomics.
- This behavior deviates from standard AMM expectations, increasing the risk of user confusion, complaints, or loss of trust.
- In extreme configurations, the compounded tax may make liquidity removal economically unviable by breaking the AMM constant formula.

Additionally, the double-taxation flaw only occurs when liquidity is removed using the ETH-supporting function (`removeLiquidityETHSupportingFeeOnTransferTokens`). In this flow, the router acts as an intermediate recipient, causing an additional transfer tax to be applied. The non-ETH `removeLiquidity()` variant (which returns WBNB instead of native BNB) performs a direct pair-to-user transfer and does not trigger the same double-tax behavior.

Assets:

- NTE.sol [<https://github.com/nodemeta/nte>]

Status:

Fixed

Classification

Impact Rate:	4/5
Likelihood Rate:	4/5
Exploitability:	Independent
Complexity:	Simple
Severity:	High

Recommendations

Remediation: Adjust the taxation logic to ensure that **liquidity removal operations are taxed at most once**. it can be done by validating that the transfer tax is not

applied if one of the participants address is router, for example:

```
} else if (pancakeRouter != from && pancakeRouter != to){  
    // Just a regular P2P transfer  
    taxBps = transferTaxBps;  
}
```

Resolution:

Fixed in `363459b`, the regular transfer is now checked if the transfer did not come from the router:

```
else if (pancakeRouter != from && pancakeRouter != to) {  
    // Just a regular P2P transfer (exclude router to prevent double taxation  
    on liquidity removal)  
    taxBps = transferTaxBps;  
}
```

Evidences

PoC Double Tax on Remove Liquidity

Reproduce:

PoC steps:

- Add liquidity to the NTE/ETH pool.
- Transfer LP tokens to `user1` to enable tax.
- Calculate expected gross NTE amount based on reserves and LP share. Compute expected buy tax and transfer tax for comparison.
- Remove liquidity using `removeLiquidityETHSupportingFeeOnTransferTokens()` by `user1`. Two tax is applied on the NTE amount transferred from pair.
- Measure actual NTE received by user and tax received by treasury.

The PoC proves that liquidity removal triggers both buy and transfer taxes. Treasury receives more than a single buy tax. Actual tax equals buy tax + transfer tax. Liquidity providers are effectively double-taxed on withdrawal.

PoC code:

```
function test_PoC_DoubleTaxOnRemoveLiquidity() public {  
    uint256 nteForLiq = 500_000 * 1e18;  
    uint256 bnbForLiq = 5 ether;  
  
    // --- Add liquidity as owner ---  
    nte.approve(BSC_ROUTER, nteForLiq);  
    (, , uint256 liquidityMinted) = router.addLiquidityETH{value: bnbForLiq}(  
        address(nte),  
        nteForLiq,
```

```

    0,
    0,
    owner,
    block.timestamp + 300
);

address pair = nte.pancakePair();

// Transfer LP to user1 so user1 removes liquidity
ILPToken(pair).transfer(user1, liquidityMinted);
uint256 lpUser1 = ILPToken(pair).balanceOf(user1);

uint256 treasuryBefore = nte.balanceOf(treasury);
uint256 user1Before = nte.balanceOf(user1);

// Compute expected "gross" NTE amount that should come out of the pair f
or lpUser1
(uint112 reserve0, uint112 reserve1,) = IPancakePair(pair).getReserves();
address token0 = IPancakePair(pair).token0();
uint256 totalLpSupply = ILPToken(pair).totalSupply();
uint256 nteReserve = (token0 == address(nte)) ? uint256(reserve0) : uint2
56(reserve1);

// amountNTEOutGross = lpAmount * nteReserve / totalLpSupply
uint256 amountNteOutGross = (lpUser1 * nteReserve) / totalLpSupply;

console.log("=== Expected amounts (gross, before taxes) ===");
console.log("LP burned (user1):", lpUser1);
console.log("NTE reserve:", nteReserve);
console.log("LP total supply:", totalLpSupply);
console.log("Expected NTE out (gross):", amountNteOutGross);

// Implemented tax model (double tax on removeLiquidity)
// 1) Pair -> Router transfer is treated as BUY and taxed
uint256 expectedBuyTax = (amountNteOutGross * BUY_TAX_BPS) / BASIS_POINTS
;

uint256 amountAfterBuyTax = amountNteOutGross - expectedBuyTax;

// 2) Router -> User transfer is treated as TRANSFER and taxed
uint256 expectedTransferTax = (amountAfterBuyTax * TRANSFER_TAX_BPS) / BA
SIS_POINTS;

uint256 expectedTotalTax_Double = expectedBuyTax + expectedTransferTax;
uint256 expectedUserNet_Double = amountNteOutGross - expectedTotalTax_Dou
ble;

// Buy-only baseline
uint256 expectedTotalTax_BuyOnly = expectedBuyTax;
uint256 expectedUserNet_BuyOnly = amountNteOutGross - expectedTotalTax_Bu

```

```

yOnly;

    console.log("=== Expected taxes ===");
    console.log("Expected buy tax (pair->router):", expectedBuyTax);
    console.log("Expected transfer tax (router->user):", expectedTransferTax)
    ;

    console.log("Expected total tax (double):", expectedTotalTax_Double);
    console.log("Expected user net (double):", expectedUserNet_Double);
    console.log("Expected user net (buy-only baseline):", expectedUserNet_Buy
Only);

    // Remove liquidity as user1
    vm.startPrank(user1);
    ILPToken(pair).approve(BSC_ROUTER, lpUser1);

    router.removeLiquidityETHSupportingFeeOnTransferTokens(
        address(nte),
        lpUser1,
        0,
        0,
        user1,
        block.timestamp + 300
    );
    vm.stopPrank();

    uint256 treasuryAfter = nte.balanceOf(treasury);
    uint256 user1After = nte.balanceOf(user1);

    uint256 actualTotalTax = treasuryAfter - treasuryBefore;
    uint256 actualUserNet = user1After - user1Before;

    console.log("=== Actual results ===");
    console.log("Actual tax to treasury:", actualTotalTax);
    console.log("Actual user received:", actualUserNet);

    // 1) Prove there is more than buy-only tax taken
    assertGt(
        actualTotalTax,
        expectedTotalTax_BuyOnly,
        "Tax exceeds buy-only -> additional tax applied on removeLiquidity"
    );

    // 2) Prove the total tax matches the (buy + transfer) model
    assertEq(
        actualTotalTax,
        expectedTotalTax_Double,
        "POC: tax equals buy tax + transfer tax (double taxation)"
    );

```

```
// 3) User amount matches expected net after double taxation
assertEq(
    actualUserNet,
    expectedUserNet_Double,
    "User received equals gross - (buy + transfer tax)"
);

// Quantify loss vs buy-only baseline
console.log("User loss vs buy-only baseline:", expectedUserNet_BuyOnly -
actualUserNet);
}
```

Results:

```
[PASS] test_POC_DoubleTaxOnRemoveLiquidity() (gas: 710463)

Logs:

=== Expected amounts (gross, before taxes) ===
LP burned (user1): 1581138830084189664999
NTE reserve: 500000000000000000000000
LP total supply: 1581138830084189665999
Expected NTE out (gross): 499999999999999999683772

=== Expected taxes ===
Expected buy tax (pair->router): 4999999999999999996837
Expected transfer tax (router->user): 14849999999999999990608
Expected total tax (double): 19849999999999999987445
Expected user net (double): 480149999999999999696327
Expected user net (buy-only baseline): 494999999999999999686935

=== Actual results ===
Actual tax to treasury: 19849999999999999987445
Actual user received: 480149999999999999696327
User loss vs buy-only baseline: 14849999999999999990608
```

Files:

NTETax.t.sol

[F-2026-14942](#) - Incorrect maxSellAmount Calculation Allows Selling Up to Total Supply Amount - Medium

Description:

The **NTE** contract is an ERC-20 token that implements an anti-dump mechanism to limit the maximum amount of tokens that can be sold in a single transaction. This behavior is controlled by the `antiDumpEnabled` flag. When enabled, sell transactions are restricted based on the `maxSellPercentage` parameter, which represents the maximum allowed sell amount as a percentage of the total token supply.

According to the implementation and validation logic in the `setAntiDumpConfig()` function, `maxSellPercentage` cannot exceed a value of `100`, where `100` represents **1%** (expressed in basis points).

```
function setAntiDumpConfig(bool enabled, uint256 maxPercentage, uint256 cooldownTime) external onlyOwner {
    if (maxPercentage == 0 || maxPercentage > 100) revert DUMP_PERCENT();
    ...
}
```

However, the current calculation of `maxSellAmount` inside the `_transferWithTax()` function uses an incorrect denominator. The calculation divides by `100` instead of `10000` (basis points), which causes the sell limit to be miscalculated and effectively disables the intended restriction.

```
// _transferWithTax()
if (antiDumpEnabled && isToPair && !taxExempt[from]) {
    uint256 maxSellAmount = (_totalSupply * maxSellPercentage) / 100;
    if (amount > maxSellAmount) revert DUMP_EXCEEDS();
    ...
}
```

For example, if `maxSellPercentage` is `100` (= 1%), the following calculation will result in `maxSellAmount = _totalSupply * 100 / 100 = _totalSupply`. As a result, the anti-dump mechanism can be bypassed, allowing users to sell up to the entire token supply in a single transaction, even when `antiDumpEnabled` is set to `true`.

This current flaw completely undermines the sell-limit protection mechanism and exposes the protocol to the following risks:

- **Anti-dump logic bypass:** Users can sell up to totalSupply in a single transaction despite configured limits.

- **Tokenomics disruption:** Large, unrestricted sell transactions may cause sudden price crashes and market instability.
- **Mismatch with documentation and expectations:** The actual behavior deviates from documented constraints and project assumptions.
- **Loss of user trust:** Holders may be exposed to unexpected dilution or price manipulation events.

Assets:

- NTE.sol [<https://github.com/nodemeta/nte>]

Status:

Fixed

Classification

Impact Rate:	4/5
Likelihood Rate:	3/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation: Consider updating the `maxSellAmount` calculation to correctly apply basis-point arithmetic by using `10000` as the denominator:

```
uint256 maxSellAmount = (_totalSupply * maxSellPercentage) / 10000;
```

Resolution: Fixed in `3ac4ad1`, the `maxSellAmount` calculation was updated to correctly apply basis-point arithmetic by using `10000` as the denominator:

```
uint256 maxSellAmount = (_totalSupply * maxSellPercentage) / 10000;
```

[F-2026-14988](#) - pancakeRouter and pancakePair Initialization Can Be Skipped in Constructor With No Recovery Mechanism - Medium

Description:

During the **NTE** contract deployment, the `_pancakeRouter` address is provided via the constructor and is expected to be used to create the primary liquidity pair (`NTE/WBNB`), with the resulting pair address stored in `pancakePair`. However, the constructor explicitly allows `_pancakeRouter` to be the zero address.

As a result, it is possible to deploy the contract with `_pancakeRouter == address(0)`. In this scenario, the deployment succeeds, but both `pancakeRouter` and `pancakePair` remain permanently set to zero addresses, as the router and pair initialization logic is intentionally skipped.

```
constructor(
    uint256 initialSupply,
    address initialOwner,
    address _treasury,
    address _pancakeRouter
) {
    ...
    if (_pancakeRouter != address(0)) {
        if (!_isContract(_pancakeRouter)) revert DEX_ROUTER();

        try IPancakeRouter(_pancakeRouter).factory() returns (address factory) {
            if (factory == address(0)) revert DEX_FACTORY_ZERO();
            if (!_isContract(factory)) revert DEX_FACTORY();

            try IPancakeRouter(_pancakeRouter).WETH() returns (address weth) {
                if (weth == address(0)) revert DEX_WETH_ZERO();
                if (!_isContract(weth)) revert DEX_WETH();

                try IPancakeFactory(factory).getPair(address(this), weth) returns
                (address existingPair) {
                    if (existingPair != address(0)) {
                        pancakePair = existingPair;
                        isPancakePair[pancakePair] = true;
                    } else {
                        address newPair = IPancakeFactory(factory).createPair(ad
                        ress(this), weth);
                        if (newPair == address(0)) revert DEX_PAIR_ZERO();
                        pancakePair = newPair;
                        isPancakePair[pancakePair] = true;
                    }
                }
            }
        }
    }
```

```

        if (pancakePair == address(0)) revert DEX_PAIR_FAIL();
        pancakeRouter = _pancakeRouter;

        // Router is NOT tax exempt to prevent arbitrage through direct router calls
    } catch {
        revert DEX_PAIR_CHECK();
    }
    } catch {
        revert DEX_WETH_CALL();
    }
    } catch {
        revert DEX_FACTORY_CALL();
    }
}
...
}

```

The **NTE** contract does not expose any setter functions to update `pancakeRouter` or `pancakePair` after deployment. Consequently, if these values are not correctly initialized in the constructor, there is no way to recover or fix the configuration without redeploying the contract.

These variables are required for certain runtime validations, including checks related to `priceImpactLimitEnabled`. If they remain unset, the corresponding logic becomes ineffective.

This can lead to:

- Any functionality depending on `pancakeRouter` or `pancakePair` variables (e.g. price impact validation) will not operate as intended.
- The contract may silently bypass or disable documented protections.
- The only remediation is redeployment, which can cause operational disruption, user confusion, and unnecessary proliferation of contract instances.

While this issue does not directly lead to fund loss, it can undermine core token mechanics and protocol guarantees.

Assets:

- NTE.sol [<https://github.com/nodemeta/nte>]

Status:

Fixed

Classification

Impact Rate: 5/5

Likelihood Rate: 3/5

Exploitability: Dependent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: Consider one of the following approaches:

- Enforce a non-zero `_pancakeRouter` address in the constructor by reverting on zero address input.
- Alternatively, introduce controlled setter functions for `pancakeRouter` and `pancakePair`, protected by appropriate access control, to allow post-deployment configuration.

Either approach would ensure that the contract can always be configured to support its documented functionality and reduce the risk of irreversible misconfiguration at deployment time.

Resolution: Fixed in `0fed2ea`, the additional setter for `pancakeRouter` and `pancakePair` was added to setup this value after the contract deployment - `setPancakeRouter()` and `setPancakePair()`.

[F-2026-15004](#) - Hardcoded Primary Pair In `_calculatePriceImpact()` Leads To Price Impact Limit Bypass On Secondary DEX Pairs - Medium

Description:

The **NTE** contract enforces a price impact limit on sells to prevent large trades from crashing the token price. When enabled, any transfer to a registered DEX pair must pass through `_calculatePriceImpact()`, which estimates the expected price movement and reverts with `PRICE_T00_HIGH()` if it exceeds `maxPriceImpactPercent` (default 500 bps / 5%).

The contract supports multiple DEX pairs. The owner registers secondary pairs (e.g., NTE/USDT) through `setDexPairStatus()`, which sets `isPancakePair[pair] = true`. The sell detection logic at line 1840 correctly uses this mapping to identify sells to any registered pair:

```
bool isToPair = isPancakePair[to];
```

The price impact gate at line 1855 then triggers for all such sells:

```
if (priceImpactLimitEnabled && isToPair && !priceImpactExempt[from] && pancakeRouter != address(0)) {
    uint256 priceImpact = _calculatePriceImpact(amount);
    if (priceImpact > maxPriceImpactPercent) revert PRICE_T00_HIGH();
}
```

However, `_calculatePriceImpact()` at line 1961 always reads reserves from the hardcoded primary pair (`pancakePair`), not from the actual destination:

```
(uint256 reserve0, uint256 reserve1,) = IPancakePair(pancakePair).getReserves();
```

The destination pair address is never passed to the function. This means a sell routed to a shallow secondary pair (e.g., 100,000 NTE liquidity) is evaluated against the deep primary pair (e.g., 10,000,000 NTE liquidity), underestimating the real impact by the ratio between the two reserves.

Regardless of whether the intended design is to protect only the primary pair or all registered pairs, the current behavior represents a requirement violation. The price impact check fires on sells to every registered pair but evaluates against only the primary pair reserves. No special or unlikely owner configuration is required to reach this state — it occurs under normal operation whenever `priceImpactLimitEnabled` is `true` and a secondary pair has been registered via `setDexPairStatus()`, both of which are standard administrative actions documented in the contract.

Assets:

- NTE.sol [https://github.com/nodemeta/nte]

Status:

Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 4/5

Exploitability: Semi-Dependent

Complexity: Simple

Severity: Medium

Recommendations

Remediation:

- Modify `_calculatePriceImpact()` to accept the destination pair address as a parameter and read reserves from that pair. The call would become `_calculatePriceImpact(amount, to)`.
- Consider restricting the price impact check to the primary pair only by replacing `isToPair` with `to == pancakePair`. This removes false protection on secondary pairs but ensures the check is accurate where it applies.

Resolution:

Fixed in `a5b8fec`. The `_calculatePriceImpact()` function now accepts an additional `pancakePair` parameter, which is used to perform the required pair-specific price impact validation.

Evidences

Price Impact Limit Bypass via Secondary Pair

Reproduce:

1. The owner deploys NTE with a primary NTE/WBNB pair holding 10,000,000 NTE.
2. The owner registers a secondary NTE/USDT pair holding 100,000 NTE via `setDexPairStatus()`.
3. The owner enables the price impact limit at 5% via `setPriceImpactLimitConfig(true, 500)`.
4. An attacker sells 50,000 NTE to the secondary pair.
5. The contract calculates 48 bps impact using primary reserves — the check passes.
6. The real impact on the secondary pair is 3,283 bps (32.8%) — over 6x the limit.

7. The transfer succeeds when it should have reverted.

Run: `forge test --match-contract PriceImpactBypassTest -vvv`

Results:

```
Compiling 1 files with Solc 0.8.28
Solc 0.8.28 finished in 3.92s
Compiler run successful!
Ran 4 tests for test/foundry/PriceImpactBypass.t.sol:PriceImpactBypassTest
[PASS] test_ExploitPriceImpactBypassViaSecondaryPair() (gas: 195769)
Logs:
Contract-calculated impact (bps): 48
Impact check: PASSES (contract thinks < 5%)
Real impact on secondary pool (bps): 3283
Real impact: SHOULD HAVE REVERTED (real > 5%)
Attacker sent (NTE): 50000
Pair received after tax (NTE): 49000
Attacker balance change (NTE): 50000
Tokens as % of secondary pool reserve: 49
=====
EXPLOIT SUCCESSFUL
=====
Price impact limit (bps): 500
Contract calculated (bps): 48
Real impact (bps): 3283
Underestimation factor: 68
Transfer succeeded -- limit was bypassed
[PASS] test_MassiveSellPassesOnSecondaryPool() (gas: 192843)
Logs:
Contract-calculated impact (bps): 421
450K NTE sell to secondary pair: PASSED (should have reverted)
Real impact on secondary pool (bps): 8147
Tokens received as % of pool reserve: 441
Secondary pool NTE price effectively destroyed
[PASS] test_PriceImpactCheckFiresOnSecondaryPair() (gas: 233704)
Logs:
Price impact check confirmed active on secondary pair sells
Same sell passes at 500 bps limit -- check fires but uses wrong reserves
[PASS] test_PrimaryPairCorrectlyBlocksHighImpactSell() (gas: 139497)
Logs:
Impact on primary pair (bps): 510
Primary pair correctly blocks high-impact sell
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 5.31ms (2.06ms CPU time)
Ran 1 test suite in 174.31ms (5.31ms CPU time): 4 tests passed, 0 failed, 0 skipped (4 total tests)
```

Files:

PriceImpactBypass.t.sol

F-2026-15018 - Buy and Sell Fees Unexpectedly Applied During Liquidity Provision Operations - Medium

Description:

The **NTE** token contract implements a taxation mechanism on token transfers, applying buy, sell, and transfer fees depending on the transaction context. These taxes are correctly applied during token purchases from a AMM, token sales to a AMM, and peer-to-peer wallet transfers.

```
function _calculateTax(address from, address to, uint256 amount) private view
returns (uint256) {
    uint256 taxBps = 0;

    if (isPancakePair[from] && !isPancakePair[to]) {
        // Buy: from DEX to user
        taxBps = buyTaxBps;
    } else if (!isPancakePair[from] && isPancakePair[to]) {
        // Sell: from user to DEX
        taxBps = sellTaxBps;
    } else if (isPancakePair[from] && isPancakePair[to]) {
        // Pool-to-pool move (usually arbitrage)
        taxBps = sellTaxBps;
    } else {
        // Just a regular P2P transfer
        taxBps = transferTaxBps;
    }

    if (taxBps == 0) return 0;

    // Check for math overflow before we calculate
    if (amount > type(uint256).max / taxBps) revert TXN_OVERFLOW();

    return (amount * taxBps) / BASIS_POINTS;
}
```

However, in the current implementation, buy and sell taxes are also applied during liquidity provision and liquidity removal operations. When users add liquidity, token transfers from the liquidity provider to the AMM pair are treated as sell transactions. Similarly, when liquidity is removed, transfers from the pair back to the user are treated as buy transactions. As a result, liquidity deposit and withdrawal flows are unintentionally taxed.

Liquidity providers expect to earn trading fees from AMM participation, not incur additional token taxes during liquidity management operations. Under the current logic, a liquidity provider depositing a large position (e.g.,

\$50,000 worth of NTE paired with \$50,000 worth of ETH) may incur a sell tax on deposit and a buy tax on withdrawal.

For example, with a 2.5% tax applied on both deposit and withdrawal, the effective NTE position becomes $\$50k * 0.975 * 0.975 = \sim\$47k$. This represents a significant and unexpected loss for liquidity providers. Such behavior may discourage liquidity provisioning, leading to reduced pool depth, higher price volatility, and degraded trading conditions for all users.

Assets:

- NTE.sol [<https://github.com/nodemeta/nte>]

Status:

Mitigated

Classification**Impact Rate:**

3/5

Likelihood Rate:

4/5

Exploitability:

Independent

Complexity:

Simple

Severity:

Medium

Recommendations**Remediation:**

Consider adjusting the tax logic to exclude liquidity provision and liquidity removal operations from buy and sell tax application. This can be achieved by detecting transfers involving the AMM pair that originate from or are destined for liquidity management functions.

Alternatively, consider providing a dedicated liquidity provision mechanism, such as wrapper functions around the PancakeSwap Router's liquidity add/remove operations, that explicitly bypass tax accumulation for liquidity providers.

Both approaches would help preserve liquidity provider incentives while maintaining the intended taxation model for regular trading activity.

Resolution:

The client is aware of the behavior and plans to exempt both the owner and the liquidity manager from taxation to ensure that liquidity provisioning from these accounts is not subject to fees. Regular users will remain subject to the standard tax rules during liquidity-related transactions.

[F-2026-15022](#) - Owner Rights Can Be Renounced While Contract Is Paused Permanently Locking Transfer - Medium

Description:

The **NTE** contract includes pausable functionality, allowing the owner to pause transfers in emergencies. The `_transferWithTax()` function implement check of the `_paused` variable status to prevent any transfer when the contract is paused.

```
function _transferWithTax(address from, address to, uint256 amount) internal
nonReentrant {

    // If we're paused, everything stops (unless you're the owner)
    if (_paused) {
        if (pauseIncludesOwner) {
            revert SYS_DISABLED();
        } else {
            if (from != _owner && to != _owner) revert SYS_DISABLED();
        }
    }
    ...
}
```

It also exposes a `renounceOwnership()` function that allows the owner to renounce ownership after 30 days from deployment, permanently setting the owner to the zero address. However, the function lacks a safeguard to prevent `renounceOwnership()` from being executed while the contract is paused.

```
function renounceOwnership() public onlyOwner {
    if (block.timestamp <= launchTime + 30 days) revert AUTH_LOCKED();
    address previousOwner = _owner;
    _owner = address(0);
    emit OwnershipTransferred(previousOwner, address(0));
}
```

If `renounceOwnership()` is called while the contract is paused, ownership rights are permanently renounced with no possibility of recovery. Since no entity will retain the ability to unpause the contract, all transfers will remain frozen indefinitely. This effectively renders the protocol unusable and permanently locks all user funds. This creates a critical risk of permanent contract paralysis, leading to severe financial loss for participants and a significant loss of trust in the protocol.

Assets:

- NTE.sol [<https://github.com/nodemeta/nte>]

Status:**Fixed****Classification****Impact Rate:** 5/5**Likelihood Rate:** 3/5**Exploitability:** Dependent**Complexity:** Simple**Severity:** **Medium****Recommendations****Remediation:**

Introduce a proper validation in the `renounceOwnership()` function to ensure ownership cannot be renounced while the contract is paused. This safeguard guarantees that the contract can always be unpaused before ownership is permanently removed.

```
function renounceOwnership() public onlyOwner {  
    if (_paused) revert SYS_DISABLED();  
    ...  
}
```

Resolution:

Fixed in `318db5a`, the pause state is now checked in the `renounceOwner()` to prevent calling function once the contract is paused:

```
function renounceOwnership() public onlyOwner {  
    if (_paused) revert SYS_DISABLED();  
    ...  
}
```

F-2026-14955 - Missing Expiration Deadline In TransactionFrom Signature Leads To Indefinitely Valid Signed Authorizations - Low

Description:

The `TransactionFrom()` function implements a meta-transaction pattern where an authorized backend signer produces a signature over transfer parameters, and a relayer submits the transaction on-chain. The signed message hash on line 837 binds the authorization to the contract address, sender, recipient, amount, category, transaction reference, nonce, and chain ID:

```
bytes32 messageHash = keccak256(
    abi.encodePacked(address(this), from, to, amount, category, txReference,
        nonce, _deploymentChainId)
);
```

No deadline or expiration timestamp is included in the signed payload. Once a backend signer produces a valid signature, that signature remains executable indefinitely — constrained only by the sequential nonce for the `from` address not having advanced and the `msg.sender` holding sufficient allowance.

The contract provides no mechanism to cancel or invalidate a pending signature. The `userCategorizedNonce` mapping is only incremented upon successful execution of a `TransactionFrom()` call (line 888). There is no standalone function to advance the nonce without executing a transfer. The only options available to a `from` address to prevent execution of a stale signature are revoking allowance from all potential relayers — an imprecise measure that may disrupt other operations — or executing another categorized transfer to advance the nonce past the stale signature.

This design diverges from established patterns for signed authorizations. [EIP-2612](#) (permit), for example, includes a `deadline` parameter specifically to bound the validity window of off-chain signatures.

In the context of a categorized business payment system — where categories, invoice references, and memos indicate commercial use — an indefinitely valid payment authorization creates operational risk. A signature generated for an invoice that is later disputed, cancelled, or paid through other means remains executable. If the `from` address has not performed another categorized transfer (keeping the nonce unchanged), and the relayer retains the signature and still holds allowance, the stale payment can be executed at any future point regardless of whether the underlying business obligation still exists.

Assets:

- NTE.sol [<https://github.com/nodemeta/nte>]

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Exploitability: Independent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Include a `deadline` parameter in both the function signature and the signed message hash. The function would then validate `block.timestamp <= deadline` before proceeding, following the pattern established by EIP-2612:

```
bytes32 messageHash = keccak256(
    abi.encodePacked(address(this), from, to, amount, category, txReference,
    nonce, deadline, _deploymentChainId)
);
require(block.timestamp <= deadline, "SIGNATURE_EXPIRED");
```

Consider implementing an `invalidateNonce()` function that allows a `from` address to advance their own nonce, effectively cancelling any pending signature tied to the current or prior nonce values without requiring execution of a transfer.

Resolution: Fixed in commit `8246c66`. An additional `deadline` parameter has been introduced and incorporated into the function logic. It is now validated as part of the signature expiry check and included in the signature hash construction.

F-2026-14943 - Floating Pragma - Info

Description:

In Solidity development, the pragma directive specifies the compiler version to be used, ensuring consistent compilation and reducing the risk of issues caused by version changes. However, using a floating pragma (e.g., `^0.8.28`) introduces uncertainty, as it allows contracts to be compiled with any version within a specified range. This can result in discrepancies between the compiler used in testing and the one used in deployment, increasing the likelihood of vulnerabilities or unexpected behavior due to changes in compiler versions.

The **NTE** contract in the current codebase use floating pragma declarations (`^0.8.28`). This increases the risk of deploying with a compiler version different from the one tested, potentially reintroducing known bugs from older versions or causing unexpected behavior with newer versions. These inconsistencies could result in security vulnerabilities, system instability, or financial loss. Locking the pragma version to a specific, tested version is essential to prevent these risks and ensure consistent contract behavior.

Assets:

- NTE.sol [<https://github.com/nodemeta/nte>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	2/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

It is recommended to lock the pragma version to the specific version that was used during development and testing. This ensures that the contract will always be compiled with a known, stable compiler version, preventing unexpected changes in behavior due to compiler updates. For example, instead of using `^0.8.xx`, explicitly define the version with `pragma solidity 0.8.28;`.

Before selecting a version, review known bugs and vulnerabilities associated with each Solidity compiler release. This can be done by referencing the official Solidity compiler release notes: [Solidity GitHub releases](#) or [Solidity Bugs by Version](#). Choose a compiler version with a good track record for stability and security.

Resolution:

Fixed in `bf38e5a`, the pragma version has been locked to `0.8.28`.

[F-2026-14953](#) - Semantically Incorrect Error Codes Across Multiple Functions Lead To Misleading Revert - Info

Description:

The **NTE** contract defines a structured custom error system with descriptive prefixes (`ADDR_*`, `CAT_*`, `MEV_*`, `CD_*`, etc.), each mapping to a specific validation domain. Thirteen instances exist where the reverted error code does not semantically match the validation it guards, degrading the diagnostic value of the error system for off-chain tooling and integrators.

The most prevalent case is `ADDR_INVALID` — defined as "Address is invalid or zero" — being used in nine locations for non-address validations:

```
// String length checks in TransactionFrom() (L850-851), updateCategoryName()
(L927-928), addCategory() (L940-941)
if (bytes(txReference).length > MAX_STRING_LENGTH) revert ADDR_INVALID();
if (bytes(categoryName).length == 0) revert ADDR_INVALID();

// Timestamp checks in setBlacklist() (L1236), setWhitelist() (L1264)
if (expiryTime <= block.timestamp) revert ADDR_INVALID();

// Numeric config in setVelocityLimitConfig() (L1422)
if (maxTx == 0 || timeWindow == 0) revert ADDR_INVALID();
```

Additional mismatches include `CD_TOO_HIGH` ("Cooldown is over 1 day") used for velocity limit bounds in `setVelocityLimitConfig()`, `DEX_ROUTER` ("Router is not a contract") used for pair validation in `setDexPairStatus()`, and `MEV_TIME_HIGH` ("Time window set too high") used to enforce that at least one parameter is non-zero in `setMevProtectionConfig()`.

The impact is limited to misleading revert diagnostics, increasing debugging time and causing incorrect error messages in frontends that parse revert reasons programmatically.

Assets:

- NTE.sol [<https://github.com/nodemeta/nte>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 2/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

- Introduce correct error codes for each mismatched validation: e.g., `CAT_NAME_INVALID` for string checks, `BL_EXPIRY_INVALID` / `WL_EXPIRY_INVALID` for timestamp checks, `MEV_VELOCITY_CONFIG` for velocity parameters, and `MEV_CONFIG_INVALID` for the zero-parameter guard.
- Replace the `DEX_ROUTER` reuse on line 1528 with `DEX_PAIR_CHECK` or a new `DEX_PAIR_NOT_CONTRACT` error, eliminating the need for the inline comment explaining the mismatch.
- Consider doing a comprehensive review of all 24 custom errors against their usage sites to ensure full semantic consistency.

Resolution: Fixed in commit `5f5812a` with new semantically correct error codes.

[F-2026-14956](#) - Missing Early Input Validation In TransactionFrom Leads To Unnecessary Gas Consumption On Invalid Calls - Info

Description:

The `TransactionFrom()` function validates the `from` parameter against the zero address at the function entry, but defers validation of `to` and `amount` to `_transferWithTax()`, which is the last operation in the execution path. Before reaching that validation, a call with `to == address(0)` or `amount == 0` executes the full signature verification pipeline: two `keccak256` hash computations, the `_recoverSigner()` call containing the `ecrecover` precompile invocation at approximately 3,000 gas, three storage reads for signer and category validation, string length checks, and the `_spendAllowance()` call involving a storage read and storage write to deduct the allowance. Only after all of these operations does execution enter `_transferWithTax()`, where the function reverts with `TXN_AMOUNT_ZERO` or `ADDR_TO_ZERO`.

```
function TransactionFrom(
    address from, address to, uint256 amount, ...
) external returns (bool) {
    if (from == address(0)) revert ADDR_FROM_ZERO();
    // 'to' and 'amount' not validated here

    // ... nonce check, keccak256 x2, ecrecover, SLOAD x3, _spendAllowance (S
    LOAD + SSTORE) ...

    _spendAllowance(from, msg.sender, amount);
    _transferWithTax(from, to, amount);
    // ^^ 'to' and 'amount' validated HERE, after 9+ operations
}
```

The impact results in increased gas costs, as unnecessary intermediate operations are executed before the transaction ultimately reverts due to invalid input.

Assets:

- NTE.sol [<https://github.com/nodemeta/nte>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 2/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Consider adding the `to` and `amount` validation to the top of `TransactionFrom()`, immediately after the existing `from` check, following the established fail-fast pattern:

```
if (from == address(0)) revert ADDR_FROM_ZERO();  
if (to == address(0)) revert ADDR_TO_ZERO();  
if (amount == 0) revert TXN_AMOUNT_ZERO();
```

Resolution: Fixed in commit `64f32c2` by adding early validation.

F-2026-14989 - Unused Event and Custom Error Declaration - Info

Description: There are some redundant event and custom error declarations that are never used within the codebase:

```
event NameSymbolUpdated(string newName, string newSymbol);  
error SYS_PAUSED();
```

Unused code contributes to unnecessary bytecode size, potentially increasing deployment and execution costs. It can also reduce code clarity and maintainability over time. While the impact may be minor, it contributes to reduced optimization and could be misleading to users.

Assets:

- NTE.sol [<https://github.com/nodemeta/nte>]

Status: Fixed

Classification

Impact Rate: 1/5
Likelihood Rate: 1/5
Exploitability: Independent
Complexity: Simple
Severity: Info

Recommendations

Remediation: Consider removing unused events and custom errors to keep the codebase clean and optimized. Alternatively, if these components are intended to be used, ensure they are implemented in appropriate locations where they add meaningful validation or observability.

Resolution: Fixed in `ebf500b`, the unused `SYS_PAUSED()` error has been removed. The previously unused event has been implemented through the introduction of the new `setNameAndSymbol()` function.

F-2026-14990 - Lack of Event Emitting for Key State Changes - Info

Description:

Several functions within the **NTE** contract perform meaningful state changes and token movements without emitting events, which hinders off-chain observability:

- `emergencyWithdrawBNB()` - perform emergency withdraw of the native asset from the **NTE** contract address;
- `setStakingContract()` - setting the address of the staking contract;
- `lockFromStaking()` - lock part of tokens for staking purposes;
- `unlockFromStaking()` - unlock funds from staking circulation.

The absence of an event makes it difficult to track admin-specific actions off-chain, which reduces transparency and complicates monitoring and auditing. It is recommended to emit events for this operation to ensure proper visibility, support off-chain indexing, and facilitate debugging, especially for critical administrative and user-facing fund transfers.

Assets:

- NTE.sol [<https://github.com/nodemeta/nte>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	2/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

Consider emitting events within the mentioned affected functions to enhance transparency and ensure reliable off-chain tracking of key state changes. Emitting events not only improves monitoring of critical storage variable updates and key operation execution but also supports indexing services, facilitates auditing, and provides better visibility into administrative actions.

Resolution:

Fixed in [e1521af](#), missing event were added for the mentioned components.

F-2026-14992 - Redundant Assignment of Default Values - Info

Description:

The **NTE** contract explicitly initializes several state variables in the constructor:

```
constructor(  
    ...  
) {  
    antiDumpEnabled = false;  
    sellCooldown = 0;  
    whitelistEnabled = false;  
    totalCategories = 0;  
    priceImpactLimitEnabled = false;  
    walletCooldownEnabled = false;  
    velocityLimitEnabled = false;  
}
```

However, since `0` and `false` are the default values for `uint256` and `bool` types respectively, the assignments to the variables mentioned above are redundant. Removing these unnecessary assignments would reduce gas costs during contract deployment and improve code efficiency without affecting functionality.

Assets:

- NTE.sol [<https://github.com/nodemeta/nte>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

It is recommended to remove the redundant assignments of default values to the mentioned variables. This will reduce the gas costs during contract deployment and improve code efficiency.

Resolution:

Fixed in `9becbca`, the redundant assignments of default values were removed.

F-2026-14993 - Inconsistent Usage of Naming Conventions - Info

Description:

Some components in the **NTE** contract do not follow the recommended Solidity naming conventions. Specifically, the `TransactionFrom()` function is not using the expected `mixedCase` format, and the `transactionProcessed()` event is not following the `CapWords` convention. This behavior is inconsistent with other functions and events in the codebase, which correctly adhere to the established naming standards.

The naming recommendations are intended to improve readability and help convey meaning more clearly through the names of functions, events, and other components. Inconsistent naming conventions reduce code readability and make the codebase harder to reason about and maintain. Over time, this can lead to confusion during reviews, audits, and future development.

Assets:

- NTE.sol [<https://github.com/nodemeta/nte>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	1/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

Consider improving consistency and readability across the codebase:

- Rename `TransactionFrom()` to use `mixedCase` – `transactionFrom()`
- Rename `transactionProcessed()` to use `CapWords` – `TransactionProcessed()`

Aligning all components with the same naming conventions will improve clarity and maintainability of the contract.

Resolution:

Fixed in `8fe1cef`, the naming convention in the specified components were aligned with recommended:

```
event TransactionProcessed(...);

function transactionFrom(
    ...
) external returns (bool);
```

F-2026-14994 - State Variables Can Be Marked as Immutable - Info

Description:

In the **NTE** contract, the state variables `antiBotDuration`, `antiBotEnabled`, `launchTime`, `pancakePair` and `pancakeRouter` are initialized in the constructor and never modified during the contract's lifecycle. Despite its immutability in practice, it is currently stored in contract storage.

```
/// @notice How long the anti-bot protection lasts after launch
uint256 public antiBotDuration;

/// @notice Whether our anti-bot shields are currently up
bool public antiBotEnabled;

/// @notice When the contract was first deployed
uint256 public launchTime;

/// @notice The PancakeSwap router we talk to
address public pancakeRouter;

/// @notice Our main trading pool (NTE/WETH)
address public pancakePair;
```

Mentioned variables are accessed frequently across multiple functions. Since it is only assigned once in the constructor and does not change afterwards, keeping it in storage is inefficient. Solidity provides the `immutable` keyword, which allows such variables to be stored directly in the contract's bytecode instead of storage, resulting in significant gas savings on variable state reading.

Assets:

- NTE.sol [<https://github.com/nodemeta/nte>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	1/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

Consider declaring mentioned variables as immutable by adding the `immutable` keyword:

```
uint256 public immutable launchTime;  
address public immutable pancakePair;  
address public immutable pancakeRouter;
```

Additionally, since `antiBotEnabled` (always `true`) and `antiBotDuration` (always `3900`) are hardcoded values set in the constructor and do not depend on constructor input params, they can be declared as `constant` rather than `immutable`:

```
uint256 public constant antiBotDuration = true;  
bool public constant antiBotEnabled = 3900;
```

This change will reduce gas costs for all operations that read these variables while maintaining the same functionality, as these values are meant to remain constant after deployment.

Alternatively, if any of the mentioned variables are expected to change after contract deployment, consider implementing dedicated setter functions to handle such updates in a controlled and explicit manner.

Resolution:

Fixed in `546d969`, mentioned variables were declared as immutable and `antiBotEnabled` as constant.

[F-2026-15007](#) - Inconsistent Use Of Magic Numbers Across Tax And Protection Configuration Reduces Maintainability And Increases Risk Of Divergence - Info

Description:

The **NTE** contract defines several named constants for critical numeric thresholds — `MAX_SELL_TAX_LIMIT` (2500), `BASIS_POINTS` (10000), and `MAX_COOLDOWN` (86400) — but uses raw numeric literals for identical or related values in other locations. The most significant instance occurs in `setAllTaxBasisPoints()`, where the sell tax cap correctly references `MAX_SELL_TAX_LIMIT` at 2500, but the buy and transfer tax caps on lines 1172 and 1174 hardcode the same value `2500` as a raw literal:

```
if (newBuyTaxBps > 2500) revert TAX_BUY_HIGH(); // L11
72 - magic number
if (newSellTaxBps > MAX_SELL_TAX_LIMIT) revert TAX_SELL_HIGH(); // L11
73 - named constant
if (newTransferTaxBps > 2500) revert TAX_XFER_HIGH(); // L11
74 - magic number
if (newBuyTaxBps + newSellTaxBps + newTransferTaxBps > 5000) ... // L11
75 - magic number
```

The same function also applies a per-change delta limit of `250` basis points three times (lines 1182-1184) without a named constant. This pattern extends beyond tax configuration:

- `setPriceImpactLimitConfig()` compares against raw `10000` on line 1311 instead of the existing `BASIS_POINTS` constant;
- `setVelocityLimitConfig()` uses `86400` on line 1425 instead of the existing `MAX_COOLDOWN`;
- `renounceOwnership()` and `emergencyWithdrawBNB()` both independently compute `launchTime + 30 days` (lines 1113 and 1375) without a shared constant.

The protection subsystem contains additional undeclared limits:

- `setMevProtectionConfig()` caps `maxBlocks` at `10` and `minTime` at `300` (lines 1391-1392);
- `_transferWithTax()` enforces a hardcoded 60-second hold-before-sell threshold (line 1797);
- `setAntiDumpConfig()` validates `maxPercentage` against `100` on line 1296 — the same `100` used as a divisor in the sell amount calculation on line 1837.

This last case (function `setAntiDumpConfig()`) also introduces a unit system inconsistency: anti-dump uses simple percentages (0-100) while all other rate parameters use basis points (0-10000).

The primary risk is that a future update to `MAX_SELL_TAX_LIMIT` would change only the sell tax cap while silently leaving buy and transfer caps at the old value, creating an unintended asymmetry. The same applies to any threshold that duplicates an existing constant. The identified inconsistencies reflect reduced code quality caused by structural asymmetry, negatively impacting readability, consistency, and long-term maintainability.

Assets:

- NTE.sol [<https://github.com/nodemeta/nte>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	1/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

Extract all tax-related limits into named constants alongside the existing `MAX_SELL_TAX_LIMIT`:

```
uint256 private constant MAX_BUY_TAX_LIMIT = 2500;  
uint256 private constant MAX_TRANSFER_TAX_LIMIT = 2500;  
uint256 private constant MAX_TOTAL_TAX_LIMIT = 5000;  
uint256 private constant MAX_TAX_CHANGE_DELTA = 250;
```

Alternatively, consider avoiding separate constants such as `MAX_BUY_TAX_LIMIT` and `MAX_TRANSFER_TAX_LIMIT`. Since all tax limits share the same maximum value (2500), a single `MAX_TAX_LIMIT` constant can be used instead.

Replace raw 10000 on line 1311 with the existing `BASIS_POINTS` constant and raw 86400 on line 1425 with the existing `MAX_COOLDOWN`.

Introduce a shared constant for the 30-day ownership lock period used on lines 1113 and 1375:

```
uint256 private constant OWNERSHIP_LOCK_PERIOD = 30 days;
```

Extract protection-specific limits into named constants:

```
uint256 private constant MAX_MEV_BLOCKS = 10;  
uint256 private constant MAX_MEV_MIN_TIME = 300;  
uint256 private constant MIN_HOLD_BEFORE_SELL = 60;  
uint256 private constant MIN_PRICE_IMPACT_BPS = 10;
```

Resolution:

Fixed in commit [44d16a2](#) with replacement of all identified magic numbers with named constants.

F-2026-15020 - Public Function not Used Internally can be Marked External - Info

Description:

There are some functions within the **NTE** codebase that are currently marked as `public`, but are not invoked internally within the contract itself. Since these function is only intended to be called externally, it should be marked as `external` instead of `public`.

Affected functions:

- `name()`
- `symbol()`
- `decimals()`
- `paused()`
- `owner()`
- `tokenURI()`
- `getCategorizedNonce()`
- `getTokenInfo()`
- `getTaxConfiguration()`
- `renounceOwnership()`
- `increaseAllowance()`
- `decreaseAllowance()`

In Solidity, functions that are only meant to be invoked by external actors, such as external contracts or users, should be designated as `external` rather than `public`. While both `public` and `external` functions are accessible from outside the contract, there are key differences. A `public` functions can be called both from other functions inside the same contract and from outside transactions, while an `external` function can only be called from outside the contract. Specifically, `external` functions are more gas-efficient when called externally, as they do not require the extra internal checks that `public` functions do. Marking a function as `public` instead of `external` introduces unnecessary gas costs.

Assets:

- NTE.sol [<https://github.com/nodemeta/nte>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Consider changing visibility to `external` instead of `public` to enhance gas efficiency and reduce unnecessary exposure for the specified functions. This change ensures that the function is only callable from external sources, reducing internal contract complexity and improving security.

Resolution: Fixed in `e25c068`. The visibility of the affected functions has been changed from `public` to `external`.

F-2026-15021 - Lack of Two Step Ownership Transfer Mechanism -

Info

Description:

The contract currently employs a basic ownable design, allowing ownership to be transferred in a single step. While this approach is simple and user-friendly, it carries a risk: if the new owner's address is entered incorrectly, ownership could be permanently transferred to an unintended or inaccessible address.

```
function transferOwnership(address newOwner) external onlyOwner {
    if (newOwner == address(0)) revert AUTH_ZERO_OWNER();
    if (newOwner == _owner) revert AUTH_SAME_OWNER();
    address previousOwner = _owner;
    _owner = newOwner;
    emit OwnershipTransferred(previousOwner, newOwner);
}
```

A safer alternative is implementing a two-step ownership transfer mechanism, often referred to as "claimable" ownership. In this model, the current owner initiates the transfer, but the designated new owner must actively confirm and claim ownership. This process adds a layer of security by ensuring the accuracy of the new owner's address and confirming their readiness to assume control of the contract.

Assets:

- NTE.sol [<https://github.com/nodemeta/nte>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 2/5

Exploitability: Dependent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

To improve the security of contract's ownership management, consider implementing the two step ownership transfer similar to the `Ownable2Step`

contract from OpenZeppelin. This contract implements a two-step process for transferring ownership, requiring the new owner to actively confirm acceptance through a contract call. This mechanism helps prevent the unintended transfer of ownership to an incorrect address, thereby enhancing overall security with an additional verification step.

Resolution:

Fixed in `8dbe695`, the `_pendingOwner` variable has been introduced. The current owner must first set the `pendingOwner`, and the designated `pendingOwner` must explicitly accept the ownership to complete the transfer process:

```
function acceptOwnership() external {
    if (msg.sender != _pendingOwner) revert AUTH_NOT_PENDING_OWNER();
    address previousOwner = _owner;
    _owner = _pendingOwner;
    _pendingOwner = address(0);
    emit OwnershipTransferred(previousOwner, _owner);
}
```

F-2026-15023 - Redundant receive Function - Info

Description:

The **NTE** contract includes a `receive()` function to allow it to accept **BNB**. The `receive()` function serves no operational purpose in this contract, as **NTE** does not use **BNB** for its logic (it is a standard **ERC20** token). Including it allows users to accidentally send **BNB** to the contract, requiring manual administrative intervention (`emergencyWithdrawBNB()`) to recover funds.

```
receive() external payable {  
    // Just log it so we know where it came from  
    emit BNBReceived(msg.sender, msg.value);  
}
```

Users who accidentally copy-paste the contract address and send **BNB** will have their funds stuck until an admin intervenes.

Assets:

- NTE.sol [<https://github.com/nodemeta/nte>]

Status:

Accepted

Classification

Impact Rate:	2/5
Likelihood Rate:	1/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

Consider removing the `receive()` function. The `emergencyWithdrawBNB()` function will still work for any **BNB** that finds its way into the contract via forced means, but the contract will correctly reject standard **BNB** transfers, giving immediate feedback (`revert`) to users who make a mistake.

Resolution:

Accepted, the client is aware and accepting the finding.

[F-2026-15041](#) - Missing msg.sender Validation in _transferWithTax - Info

Description:

The `_transferWithTax()` function applies all access control and policy checks only to the transfer endpoints (`from` and `to`). The transaction initiator (`msg.sender`) is not validated in any of these checks.

As a result, any address that has been granted allowance can trigger transfers that satisfy the endpoint conditions, even if the initiator itself would not satisfy the intended restriction. The logic therefore enforces rules based solely on which addresses appear as sender or receiver, without restricting who is actually initiating the transfer.

This pattern affects multiple control mechanisms inside the function:

- During pause (when `pauseIncludesOwner == false`), transfers involving the owner are allowed. However, the code does not require the caller to be the owner. Any approved spender can move owner tokens during pause via `transferFrom()`, which weakens the intended emergency-stop behavior.

```
if (_paused) {
    if (pauseIncludesOwner) {
        revert SYS_DISABLED();
    } else {
        if (from != _owner && to != _owner) revert SYS_DISABLED();
    }
}
```

- During the anti-bot window, transfers are allowed if either side is owner or tax-exempt. Since the initiator is not validated, a non-exempt address with allowance from an exempt address can still move tokens during the protected launch period.

```
// Launch day shields - very strict for the first hour or so
if (antiBotEnabled && block.timestamp < launchTime + antiBotDuration) {
    if (!(from == _owner || to == _owner || taxExempt[from] || taxExempt[to]))
    {
        revert SEC_BOT_ACTIVE();
    }
}
```

- Blacklist enforcement blocks transfers only when `from` or `to` is blacklisted. A blacklisted address can still initiate `transferFrom()`

between two non-blacklisted accounts and move their tokens. The blacklist therefore prevents direct participation but does not prevent operational involvement.

```
if (isBlacklistedActive(from)) revert BL_SENDER();
if (isBlacklistedActive(to)) revert BL_RECIPIENT();
```

- When whitelist mode is enabled, transfers are permitted if one side is whitelisted (or owner/contract). A non-whitelisted address that has allowance from a whitelisted address can pull tokens to itself. This allows non-whitelisted users to receive and control tokens while whitelist mode is active.

```
if (whitelistEnabled) {
    if (!(from == _owner || to == _owner ||
        isWhitelistedActive(from) || isWhitelistedActive(to) ||
        from == address(this) || to == address(this))) {
        revert WL_REQUIRED();
    }
}
```

The impact depends on the intended design. If the goal was to restrict which addresses may appear as sender or receiver, the current implementation is consistent. However, if the intent was to restrict who may initiate transfers under pause, whitelist, blacklist, anti-bot, the implementation does not achieve that objective. In its current form, policy enforcement applies to endpoints only, not to the actor triggering the transaction.

To align behavior with stricter access control expectations, `msg.sender` should be validated in the relevant branches. Depending on the intended model, this may include preventing blacklisted initiators from calling `transferFrom`, requiring privileged initiators during pause or anti-bot windows. If the current behavior is intentional, it should be clearly documented together with the trust assumptions regarding allowances and relayer usage.

Assets:

- NTE.sol [<https://github.com/nodemeta/nte>]

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: If the intended security model requires restricting **who may initiate transfers**, introduce explicit `msg.sender` validation in relevant branches mentioned.

If endpoint-only enforcement is intentional, document this clearly and define trust assumptions around allowances and relayer usage.

Resolution: Fixed in `2b2bdc3`, explicit `msg.sender` validation has been introduced in the mentioned branches.

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.

The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at <https://hacken.io/audits/>.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

Appendix 1. Definitions

Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

[hkenio/severity-formula](https://github.com/hackenio/severity-formula)

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.

Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details	
Repository	https://github.com/nodemeta/nte
Initial Commit	04d20e37cacc93e42945ac691a297688545e9b3a
Final Commit	58ae668b2b8075c0d5f6a54f87036a464e305068
Whitepaper	N/A
Requirements	README.md
Technical Requirements	README.md

Asset	Type
NTE.sol [https://github.com/nodemeta/nte]	Smart Contract

Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.

Frameworks and Methodologies

This security assessment was conducted in alignment with recognised penetration testing standards, methodologies and guidelines, including the [NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment](#), and the [Penetration Testing Execution Standard \(PTES\)](#). These assets provide a structured foundation for planning, executing, and documenting technical evaluations such as vulnerability assessments, exploitation activities, and security code reviews. Hacken's internal penetration testing methodology extends these principles to Web2 and Web3 environments to ensure consistency, repeatability, and verifiable outcomes.